PostgreSQL — Zero to Master

22 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (6 topics)

01 Introduction and Installation

02 Databases, Schemas and psql Commands

03 Data Types and Creating Tables

04 CRUD and RETURNING

05 Filtering, Sorting, Pagination

06 Constraints

Intermediate (6 topics)

07 Joins and Set Operations

08 Aggregates, GROUP BY and FILTER

09 Subqueries and CTEs (Recursive)

10 Window Functions

11 JSONB and Arrays

12 Views and Materialized Views

Advanced (6 topics)

13 Indexes: B-tree, GIN, GiST, BRIN

14 Transactions, MVCC and Locking

15 Functions, Procedures and Triggers (PL/pgSQL)

16 Full-text Search

17 Roles, Permissions and Row-Level Security

18 Extensions

Master (4 topics)

19	Performance Tuning and VACUUM
20	Backup, Restore and PITR
21	Replication, HA and Partitioning
22	Real Project: Multi-tenant SaaS Schema

Beginner Level

01. Introduction and Installation

DEFINITION

PostgreSQL is a powerful open-source object-relational database known for standards compliance, extensibility and reliability. Use `psql` or pgAdmin.

SYNTAX

```
psql -U postgres
```

EXAMPLE

```
psql -U postgres
\l
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A heavy-duty accounting office that can handle complicated books and special records; PostgreSQL is a powerful, standards-focused database.

தமிழ்

சிக்கலான கணக்குகளையும் சிறப்புப் பதிவுகளையும் கையாளும் வலுவான கணக்கு அலுவலகம் போல PostgreSQL ஒரு சக்திவாய்ந்த database.

PROS

- Very standards compliant
- Rich features (JSONB, extensions)
- Reliable and ACID
- Free and open source

CONS

- Steeper learning curve
- Needs tuning (VACUUM)
- Slightly more complex setup than MySQL

ADVANCED / NEW FEATURES

- PostgreSQL 18: async I/O, `uuidv7()`, virtual generated columns, OAuth auth
- `MERGE` (15)
- `JSON_TABLE` and incremental backup (17)

02. Databases, Schemas and psql Commands

DEFINITION

Create databases and schemas; useful meta-commands: `\dt`, `\d table`, `\c db`, `\q`.

SYNTAX

```
CREATE SCHEMA s;
\c db
```

EXAMPLE

```
CREATE DATABASE shop;
\c shop
CREATE SCHEMA sales;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A multi-storey building: each floor (schema) holds separate shops' records inside the same building (database).

தமிழ்

பல மாடிக் கட்டிடம்: ஒவ்வொரு தளமும் (schema) ஒரே கட்டிடத்துக்குள் (database) தனித்தனி கடைகளின் பதிவுகளை வைத்திருக்கும்.

PROS

- Logical separation
- Multi-tenant designs

CONS

- More namespaces to manage

ADVANCED / NEW FEATURES

- Schema-level privileges improvements

03. Data Types and Creating Tables

DEFINITION

`integer`, `bigserial`, `numeric`, `text`, `varchar`, `boolean`, `date`, `timestamptz`, `uuid`, `jsonb`, arrays, enums.

SYNTAX

```
CREATE TABLE t (id bigserial PRIMARY KEY, data jsonb);
```

EXAMPLE

```
CREATE TABLE customers (
  id bigserial PRIMARY KEY,
```

```
  name text NOT NULL,
  created_at timestamptz DEFAULT now()
);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A register with strict columns and a free-notes column; PostgreSQL has many data types including arrays, UUID and JSONB.

தமிழ்

கட்டாய நெடுவரிசைகளும் தனிக் குறிப்பு நெடுவரிசையும் உள்ள பதிவேடு போல PostgreSQL arrays, UUID, JSONB உட்பட பல வகைகளைக் கொண்டுள்ளது.

PROS

- Rich types
- Custom types and enums

CONS

- Type conversion strictness needs care

ADVANCED / NEW FEATURES

- Identity columns
- `uuidv7()` (18)
- Virtual generated columns (18)

04. CRUD and RETURNING

DEFINITION

Standard CRUD with the handy `RETURNING` clause.

SYNTAX

```
INSERT ... RETURNING id;
```

EXAMPLE

```
INSERT INTO customers (name) VALUES ('Priya') RETURNING id;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop receipt that immediately tells your new bill number; `RETURNING` gives the created row back.

தமிழ்

புதிய பில் எண்ணை உடனே சொல்லும் ரசீது போல `RETURNING` உருவாக்கிய வரிசையை திருப்பித் தருகிறது.

PROS

- Saves an extra query
- Atomic

CONS

- PostgreSQL-specific (not standard everywhere)

ADVANCED / NEW FEATURES

- `MERGE ... RETURNING` (17)

05. Filtering, Sorting, Pagination

DEFINITION

`WHERE`, `ILIKE`, `ORDER BY`, `LIMIT/OFFSET`, and keyset pagination for large tables.

SYNTAX

```
SELECT ... WHERE x ILIKE 'a%' ORDER BY id LIMIT 20;
```

EXAMPLE

```
SELECT * FROM customers WHERE name ILIKE 'p%' ORDER BY id LIMIT 20;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Looking for names starting with 'A' in a phone book without worrying about capitals; ILIKE ignores case.

தமிழ்

பெரிய/சிறிய எழுத்து பற்றிக் கவலைப்படாமல் தொலைபேசிப் புத்தகத்தில் 'A' எனத் தொடங்கும் பெயர்களைத் தேடுவது போல ILIKE case-ஐ புறக்கணிக்கிறது.

PROS

- Case-insensitive search
- Keyset pagination possible

CONS

- OFFSET is slow for deep pages

ADVANCED / NEW FEATURES

- Better index-only scans

06. Constraints

DEFINITION

Primary, foreign, unique, check and exclusion constraints keep data valid.

SYNTAX

```
CHECK (qty > 0)
UNIQUE (email)
```

EXAMPLE

```
ALTER TABLE orders ADD CONSTRAINT qty_pos CHECK (qty > 0);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Rules at the counter: quantity cannot be negative and a ration card number cannot repeat.

தமிழ்

கவுண்டர் விதிகள்: எண்ணிக்கை எதிர்மறையாக இருக்கக்கூடாது, ரேஷன் எண் மீண்டும் வரக்கூடாது.

PROS

- Strong integrity
- Exclusion constraints for ranges

CONS

- Constraints add write cost

ADVANCED / NEW FEATURES

- `NULLS NOT DISTINCT` unique (15)
- Temporal constraints (18)

Intermediate Level

07. Joins and Set Operations

DEFINITION

All joins plus `UNION`, `INTERSECT`, `EXCEPT`; `LATERAL` joins.

SYNTAX

```
SELECT ... JOIN ... UNION ...
```

EXAMPLE

```
SELECT c.name, o.total FROM customers c LEFT JOIN orders o ON o.customer_id = c.id;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Combining the customer register with orders, or merging two lists of guests; joins and set operations combine data.

தமிழ்

வாடிக்கையாளர் பதிவேட்டை ஆர்டர்களுடன் இணைப்பது அல்லது இரண்டு விருந்தினர் பட்டியல்களை இணைப்பது போல joins, set operations தரவை இணைக்கின்றன.

PROS

- All join types
- LATERAL joins

CONS

- Complex joins need good indexes

ADVANCED / NEW FEATURES

- Better join planning
- Parallel joins

08. Aggregates, GROUP BY and FILTER

DEFINITION

Aggregates with `FILTER`, `GROUPING SETS`, `string_agg`, `array_agg`.

SYNTAX

```
SELECT count(*) FILTER (WHERE x) FROM t;
```

EXAMPLE

```
SELECT count(*) FILTER (WHERE total > 1000) FROM orders;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Counting only the paid bills out of all bills in one pass; FILTER adds conditions inside aggregates.

தமிழ்

எல்லா பில்களிலும் பணம் செலுத்தியவற்றை மட்டும் ஒரே பாஸில் எண்ணுவது போல FILTER aggregates-க்குள் நிபந்தனை சேர்க்கிறது.

PROS

- Concise reports
- `string_agg`, `array_agg`

CONS

- Syntax differs from other databases

ADVANCED / NEW FEATURES

- `GROUPING SETS`, `CUBE`, `ROLLUP`

09. Subqueries and CTEs (Recursive)

DEFINITION

CTEs simplify queries; recursive CTEs traverse trees such as categories.

SYNTAX

```
WITH RECURSIVE t AS (...) SELECT * FROM t;
```

EXAMPLE

```
WITH RECURSIVE t AS (
  SELECT id, parent_id FROM categories WHERE parent_id IS NULL
  UNION ALL
  SELECT c.id, c.parent_id FROM categories c JOIN t ON c.parent_id = t.id
) SELECT * FROM t;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Finding all branches under a regional manager, then their sub-branches; recursive CTEs walk through trees.

தமிழ்

பிராந்திய மேலாளரின் கீழ் உள்ள அனைத்து கிளைகளையும் அவற்றின் துணைக் கிளைகளையும் கண்டுபிடிப்பது போல recursive CTEs மரங்களை நடக்கின்றன.

PROS

- Hierarchical queries
- Readable steps

CONS

- Infinite recursion risk

ADVANCED / NEW FEATURES

- `SEARCH` and `CYCLE` clauses
- CTE inlining (12)

10. Window Functions

DEFINITION

`ROW_NUMBER`, `RANK`, `LAG/LEAD`, running totals.

SYNTAX

```
SELECT sum(x) OVER (ORDER BY id) FROM t;
```

EXAMPLE

```
SELECT id, total, sum(total) OVER (ORDER BY id) FROM orders;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A running total in a shop passbook that shows balance after every entry; window functions compute running values.

தமிழ்

ஒவ்வொரு பதிவுக்குப் பின்னும் இருப்பைக் காட்டும் பாஸ்புக் போல window functions ஓடும் மொத்தத்தைக் கணக்கிடுகின்றன.

PROS

- Advanced analytics
- No self-joins

CONS

- Memory heavy on big partitions

ADVANCED / NEW FEATURES

- Window function optimisations

11. JSONB and Arrays

DEFINITION

Store and query semi-structured data with `jsonb` operators and GIN indexes.

SYNTAX

```
SELECT data->>'k' FROM t WHERE data @> '{"a":1}';
```

EXAMPLE

```
SELECT data->>'city' FROM profiles WHERE data @> '{"vip": true}';
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A form with extra notes written freely; JSONB stores flexible documents with fast searching.

தமிழ்

கூடுதல் குறிப்புகளை சுதந்திரமாக எழுதும் படிவம் போல JSONB நெகிழ்வான ஆவணங்களை வேகமான தேடலுடன் சேமிக்கிறது.

PROS

- Schema flexibility inside SQL
- GIN indexing

CONS

- Poor modelling if everything is JSON

ADVANCED / NEW FEATURES

- `JSON_TABLE` (17)
- SQL/JSON constructors (16)

12. Views and Materialized Views

DEFINITION

Views store queries; materialized views store results and can be refreshed.

SYNTAX

```
CREATE MATERIALIZED VIEW m AS SELECT ...;
REFRESH MATERIALIZED VIEW m;
```

EXAMPLE

```
CREATE MATERIALIZED VIEW monthly AS SELECT date_trunc('month', created_at) m, sum(total) FROM
orders GROUP BY 1;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A monthly sales report printed once and kept for quick viewing instead of recomputing each time.

தமிழ்

ஒவ்வொரு முறையும் கணக்கிடாமல் மாதாந்திர விற்பனை அறிக்கையை ஒருமுறை அச்சிட்டு வைத்திருப்பது போல materialized views முடிவுகளைச் சேமிக்கின்றன.

PROS

- Fast heavy reports
- Indexable

CONS

- Data can be stale
- Refresh cost

ADVANCED / NEW FEATURES

- `REFRESH ... CONCURRENTLY`
- Incremental view maintenance (extension)

Advanced Level

13. Indexes: B-tree, GIN, GiST, BRIN

DEFINITION

Choose index types per workload; partial and covering indexes; verify with `EXPLAIN (ANALYZE, BUFFERS)`.

SYNTAX

```
CREATE INDEX ON t USING gin (col);
```

EXAMPLE

```
CREATE INDEX CONCURRENTLY idx_orders_cust ON orders (customer_id);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Different kinds of book indexes: by name, by topic keyword, by location; Postgres offers B-tree, GIN, GiST and BRIN indexes.

தமிழ்

பெயர், தலைப்பு, இடம் எனப் பல வகை புத்தக அட்டவணைகள் போல Postgres B-tree, GIN, GiST, BRIN indexes வழங்குகிறது.

PROS

- Right index for each workload
- Partial and covering indexes

CONS

- Extra storage
- Wrong choice slows writes

ADVANCED / NEW FEATURES

- Parallel index builds
- B-tree skip scan (18)

14. Transactions, MVCC and Locking

DEFINITION

MVCC lets readers not block writers; isolation levels; row locks (`FOR UPDATE`), advisory locks, deadlocks.

SYNTAX

```
BEGIN; SELECT ... FOR UPDATE; COMMIT;
```

EXAMPLE

```
BEGIN;
SELECT * FROM acc WHERE id = 1 FOR UPDATE;
COMMIT;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

While you change a cheque amount, nobody else edits it; MVCC lets readers continue without waiting.

தமிழ்

நீங்கள் காசோலைத் தொகையை மாற்றும்போது வேறு யாரும் திருத்த முடியாது; MVCC வாசகர்கள் காத்திருக்காமல் தொடர உதவுகிறது.

PROS

- Readers don't block writers
- Strong isolation levels

CONS

- Long transactions bloat tables
- Deadlocks

ADVANCED / NEW FEATURES

- `SKIP LOCKED` queues
- Serializable snapshot isolation

15. Functions, Procedures and Triggers (PL/pgSQL)

DEFINITION

Server-side logic with PL/pgSQL, triggers and event triggers.

SYNTAX

```
CREATE FUNCTION f() RETURNS trigger AS $$ ... $$ LANGUAGE plpgsql;
```

EXAMPLE

```
CREATE FUNCTION touch() RETURNS trigger AS $$
BEGIN NEW.updated_at = now(); RETURN NEW; END $$ LANGUAGE plpgsql;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

An automatic bell that rings whenever an entry is added; triggers and functions run server-side logic.

தமிழ்

பதிவு சேர்க்கப்படும்போது தானாக ஒலிக்கும் மணி போல triggers, functions server-side logic-ஐ இயக்குகின்றன.

PROS

- Powerful server logic
- Many procedural languages

CONS

- Hard to debug and version

ADVANCED / NEW FEATURES

- Procedures with transaction control (11)
- SQL-standard function bodies (14)

16. Full-text Search

DEFINITION

`tsvector`, `tsquery`, GIN indexes for fast search.

SYNTAX

```
to_tsvector('english', body) @@ to_tsquery('x')
```

EXAMPLE

```
SELECT * FROM posts WHERE to_tsvector('english', body) @@ to_tsquery('postgres & index');
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Searching a library's shelf by keywords instead of reading every book; full-text search finds words quickly.

தமிழ்

ஒவ்வொரு புத்தகத்தையும் படிக்காமல் முக்கிய சொற்களால் நூலகத்தைத் தேடுவது போல full-text search சொற்களை விரைவாகக் கண்டுபிடிக்கிறது.

PROS

- Built in search
- GIN indexes

CONS

- Less feature-rich than dedicated search engines

ADVANCED / NEW FEATURES

- `websearch_to_tsquery`
- `pg_trgm` fuzzy search

17. Roles, Permissions and Row-Level Security

DEFINITION

Roles and grants; Row-Level Security policies for multi-tenant data.

SYNTAX

```
ALTER TABLE t ENABLE ROW LEVEL SECURITY;
```

EXAMPLE

```
ALTER TABLE docs ENABLE ROW LEVEL SECURITY;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Each shop in a mall can see only its own accounts in a shared system; row-level security enforces that.

தமிழ்

வளாகத்தில் ஒவ்வொரு கடையும் பகிரப்பட்ட அமைப்பில் தன் கணக்குகளை மட்டும் பார்ப்பது போல row-level security அதை உறுதிப்படுத்துகிறது.

PROS

- Multi-tenant safety
- Centralised policy

CONS

- Policies can slow queries
- Complex to debug

ADVANCED / NEW FEATURES

- `SECURITY INVOKER` views (15)
- Predefined roles

18. Extensions

DEFINITION

`pg_stat_statements`, `postgis`, `pgcrypto`, `pg_trgm`, `pgvector`.

SYNTAX

```
CREATE EXTENSION pg_trgm;
```

EXAMPLE

```
CREATE EXTENSION pg_trgm;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Adding attachments to a mixer-grinder for special jobs; extensions add features like PostGIS, pgvector and pg_trgm.

தமிழ்

மிக்ஸியில் சிறப்பு வேலைகளுக்கு இணைப்புகளைச் சேர்ப்பது போல extensions PostGIS, pgvector போன்ற அம்சங்களைச் சேர்க்கின்றன.

PROS

- Huge ecosystem
- Add vectors, GIS, time-series

CONS

- Managed services restrict some extensions

ADVANCED / NEW FEATURES

- `pgvector` for AI embeddings
- `pg_stat_statements` improvements

Master Level

19. Performance Tuning and VACUUM

DEFINITION

Autovacuum, `ANALYZE`, `work_mem`, `shared_buffers`, connection pooling (PgBouncer), query plans.

SYNTAX

```
VACUUM (ANALYZE) t;
```

EXAMPLE

```
VACUUM (ANALYZE) orders;
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Cleaning the shop of old and torn bills and updating the stock list regularly; VACUUM removes dead rows and updates statistics.

தமிழ்

பழைய, கிழிந்த பில்களை அகற்றி இருப்புப் பட்டியலை புதுப்பிப்பது போல VACUUM dead rows-ஐ அகற்றி புள்ளிவிவரங்களைப் புதுப்பிக்கிறது.

PROS

- Reclaims space
- Better plans

CONS

- Needs monitoring
- Bloat if ignored

ADVANCED / NEW FEATURES

- Improved autovacuum
- Parallel vacuum
- Eager freezing (18)

20. Backup, Restore and PITR

DEFINITION

`pg_dump`, `pg_restore`, base backups and WAL archiving for point-in-time recovery.

SYNTAX

```
pg_dump -Fc db > db.dump
```

EXAMPLE

```
pg_dump -Fc shop > shop.dump
pg_restore -d shop shop.dump
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Keeping a duplicate ledger and daily copies in a locker so any day can be restored; backups and WAL enable point-in-time recovery.

தமிழ்

நகல் பதிவேடும் தினசரி பிரதிகளும் லாக்கரில் வைப்பது போல backups, WAL எந்த நாளையும் மீட்க உதவுகின்றன.

PROS

- Flexible backups
- PITR with WAL

CONS

- Needs a tested restore plan

ADVANCED / NEW FEATURES

- Incremental base backups (17)
- `pg_createsubscriber`

21. Replication, HA and Partitioning

DEFINITION

Streaming and logical replication, Patroni failover, declarative partitioning.

SYNTAX

```
CREATE TABLE t (...) PARTITION BY RANGE (d);
```

EXAMPLE

```
CREATE TABLE logs (...) PARTITION BY RANGE (created_at);
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Branch shops copy the main stock list so customers are served even if the head office is busy; replication and partitioning scale reads and big tables.

தமிழ்

தலைமை அலுவலகம் பிஸியாக இருந்தாலும் வாடிக்கையாளர்களுக்கு சேவை செய்ய கிளைகள் இருப்புப் பட்டியலை நகலெடுப்பது போல replication, partitioning scale செய்கின்றன.

PROS

- High availability
- Read scaling
- Faster queries on huge tables

CONS

- Operational complexity

ADVANCED / NEW FEATURES

- Logical replication improvements
- Failover slots (17)
- Patroni/CloudNativePG

22. Real Project: Multi-tenant SaaS Schema

DEFINITION

Schemas/RLS, indexes, migrations, reporting views and backups for a SaaS app.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shared accounting system for many shops where each shop sees only its own data; a multi-tenant SaaS schema practice project.

தமிழ்

பல கடைகளுக்கான பொதுக் கணக்கு அமைப்பில் ஒவ்வொரு கடையும் தன் தரவை மட்டும் காணும்; ஒரு multi-tenant SaaS schema பயிற்சி project.

PROS

- Real SaaS practice

CONS

- Design effort

ADVANCED / NEW FEATURES

- Use RLS + Postgres 17/18 features

Interview Question Bank

Q1. What is MVCC?
Multi-Version Concurrency Control keeps row versions so readers and writers do not block each other.

Q2. PostgreSQL vs MySQL?
PostgreSQL has richer SQL features, extensions and types (jsonb, arrays); MySQL is simpler and widely used.

Q3. What is VACUUM?
Removes dead row versions and updates statistics to reclaim space.

Q4. What index types exist?
B-tree, Hash, GIN, GiST, SP-GiST, BRIN.

Q5. `json` vs `jsonb`?
`jsonb` is binary, indexable and faster to query; `json` stores exact text.

Q6. What is a CTE?
A named temporary result set using `WITH`, possibly recursive.

Q7. What is a materialized view?
A view whose results are stored and refreshed on demand.

Q8. How do you analyse a query?
`EXPLAIN (ANALYZE, BUFFERS)` shows plan and real timings.

Q9. What is Row-Level Security?
Policies restricting which rows a role can see or change.

Q10. What is WAL?
Write-Ahead Log ensuring durability and enabling replication and recovery.

Capstone Project Ideas

1. Online shop schema
2. Analytics with window functions
3. Multi-tenant SaaS with RLS
4. Full-text blog search
5. Geo app with PostGIS